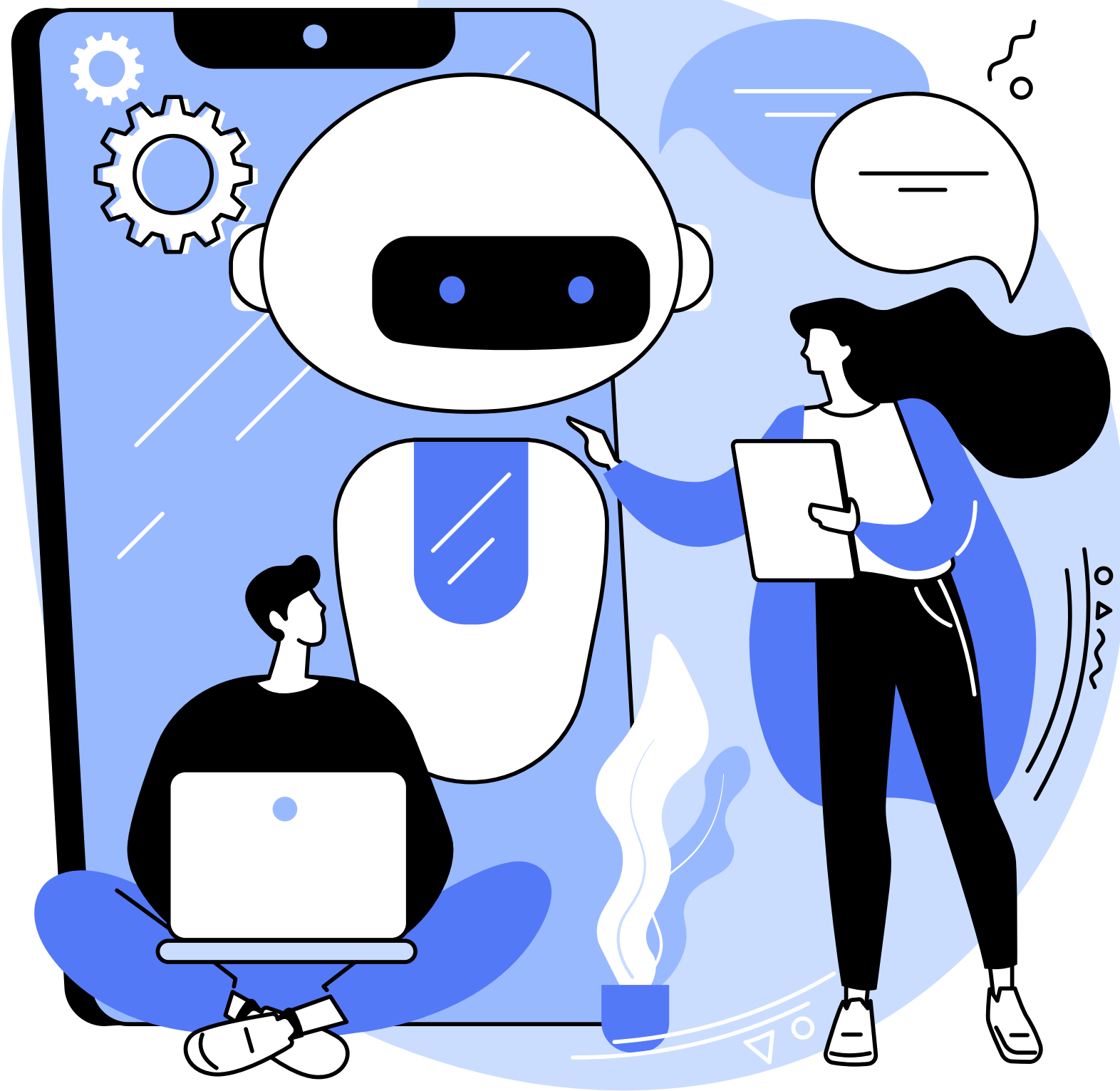




**TRUBA**  
**server**



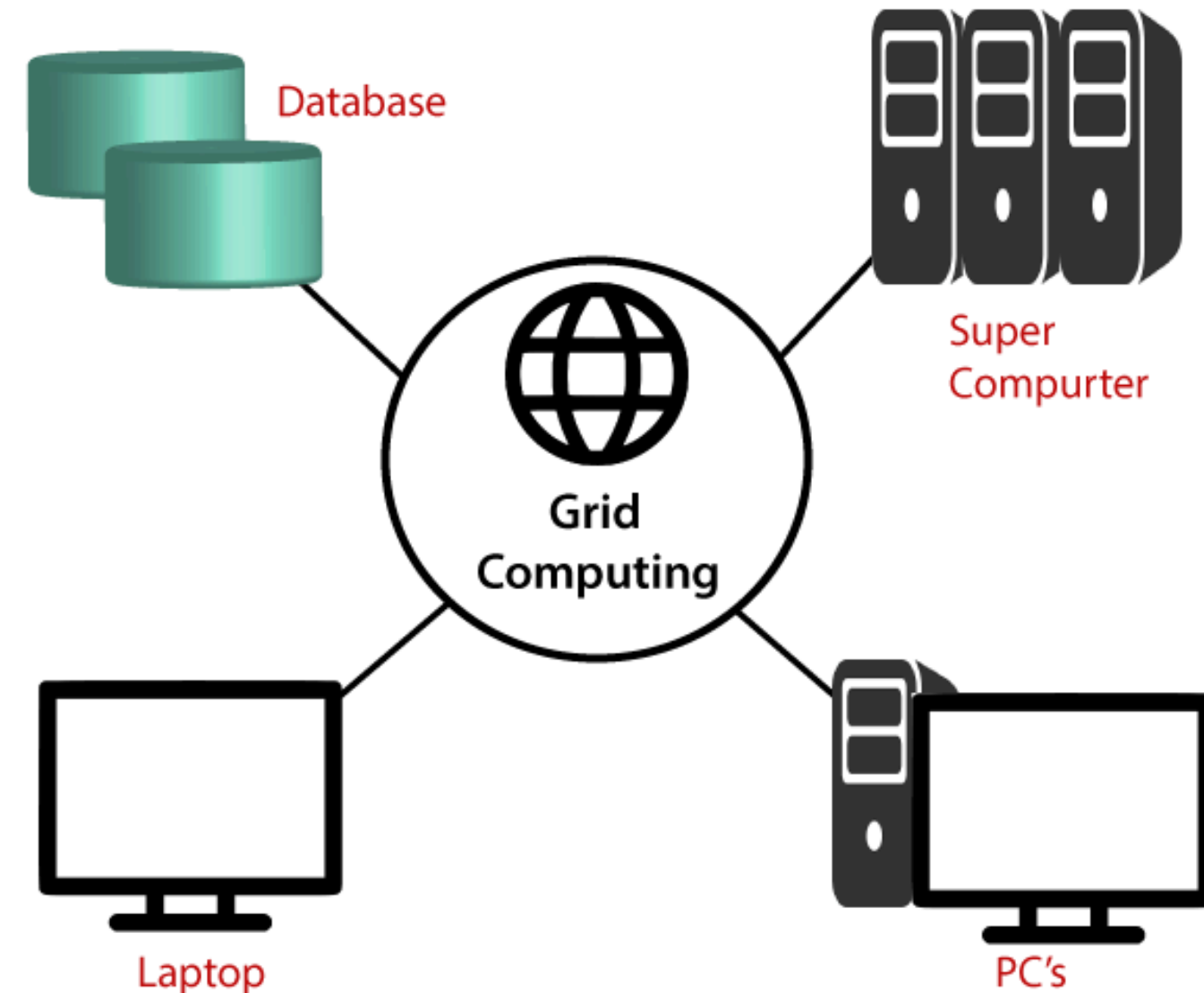
- TÜBİTAK ULAKBİM **Yüksek Başarımli ve Grid Hesaplama Merkezi**, araştırmacılara yüksek başarımli hesaplama ve veri depolama imkânı sunar.
- TRUBA (**Türk Ulusal Bilim e-Altyapısı**) adıyla anılır.
- Amaç, araştırmacıların ulusal ve uluslararası rekabet gücünü artırmaktır.

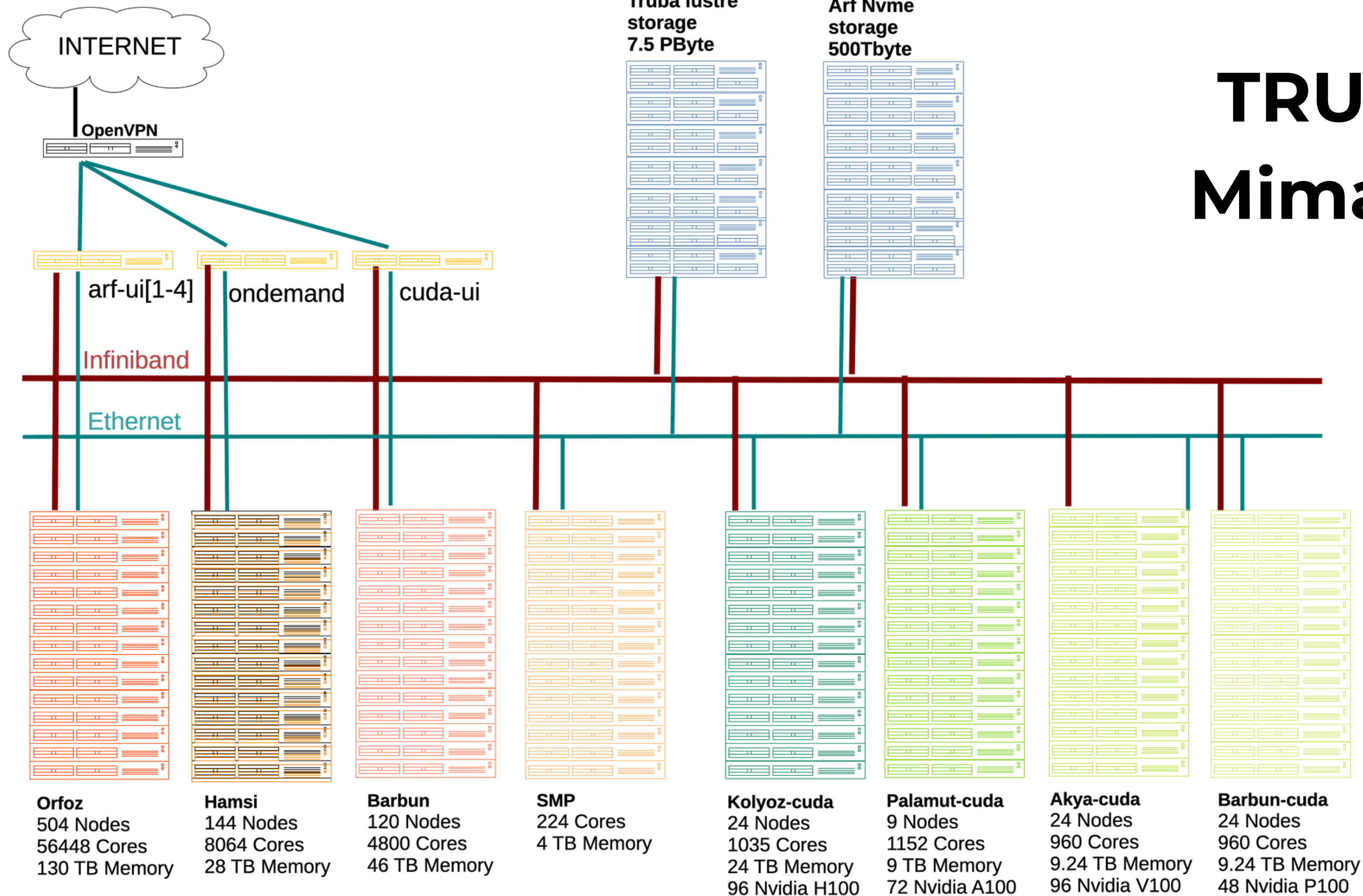
# Yüksek Başarımli Hesaplama (High Performance Computing)

Çok büyük verileri veya karmaşık bilimsel hesaplamaları, **birçok işlemciyi (CPU/GPU)** aynı anda kullanarak çok hızlı şekilde yapan sistemlerdir.

## Grid Hesaplama

Farklı yerlerde bulunan bilgisayarların ağ üzerinden birbirine bağlanarak **ortak bir süper bilgisayar** gibi çalıştığı yapıdır.





# TRUBA

## Mimarisi



# ARF

- ARF, yüksek başarımlı hesaplama (HPC) ihtiyaçları için tasarlanmış modern ve geniş ölçekli bir kümedir.
- Araştırmacılara yüksek çekirdek sayısı, hızlı ağ altyapısı ve merkezi depolama olanakları sağlar.

Bağlanmak için  
OpenVPN  
bağlantısının aktif  
olması gerekir.

```
ssh <kullanici-adiniz>@<ip-adresi>  
ssh oceylan@172.16.6.11
```



|  | Kuyruk      | Sunucu Modeli                                          | Sunucu Adet | İşlemci Modeli               | Çekirdek /Sunucu         | Toplam Bellek | GPU                        | Maksimum Süre | İşletim Sistemi |
|--|-------------|--------------------------------------------------------|-------------|------------------------------|--------------------------|---------------|----------------------------|---------------|-----------------|
|  | orfoz       | Orfoz                                                  | 504         | 2x Intel Xeon Platinum 8480+ | 112 (110 kullanılabilir) | 256 GB        | Yok                        | 3 gün         | Rocky Linux 9.2 |
|  | barbun      | Barbun                                                 | 119         | 2x Intel Xeon Gold 6248R     | 40                       | 384 GB        | Yok                        | 3 gün         | Rocky Linux 9.2 |
|  | barbun-cuda | Barbun                                                 | 24          | 2x Intel Xeon Gold 6248R     | 40                       | 384 GB        | 2x Nvidia P100 16GB        | 3 gün         | Rocky Linux 9.2 |
|  | akya-cuda   | Akya                                                   | 24          | 2x Intel Xeon Gold 6248R     | 40                       | 384 GB        | 4x Nvidia V100 16GB NVLink | 3 gün         | Rocky Linux 9.2 |
|  | hamsi       | Hamsi                                                  | 144         | 2x Intel Xeon Gold 6338      | 56 (54 kullanılabilir)   | 192 GB        | Yok                        | 3 gün         | Rocky Linux 9.2 |
|  | smp         | Orkinos                                                | 1           | 4x Intel Xeon Gold 6248R     | 224                      | 4 TB          | Yok                        | 3 gün         | Rocky Linux 9.2 |
|  | debug       | Çeşitli (orfoz, hamsi, barbun, barbun-cuda, akya-cuda) | 200         | Çeşitli                      | Çeşitli                  | Çeşitli       | Çeşitli                    | 4 saat        | Rocky Linux 9.2 |
|  |             |                                                        |             |                              |                          |               |                            |               |                 |

# GPU'lu Olanlar İçin Özel Gereksinimler

- Barbun-cuda: En az 20 çekirdek + 1 GPU, 3 gün, 384 GB RAM, 2×Nvidia P100 (16 GB)
- Akya-cuda: En az 10 çekirdek + 1 GPU, 3 gün, 384 GB RAM, 4×Nvidia V100 (16 GB NVLink), 1.4 TB NVMe disk (/tmp)
- Debug: Maksimum 4 saat, kısa test işleri



```
● (base) bash-5.1$ scontrol show partition=akya-cuda
PartitionName=akya-cuda
  AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
  AllocNodes=ALL Default=NO QoS=N/A
  DefaultTime=00:02:00 DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=3-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
  Nodes=akya[1-20]
  PriorityJobFactor=3000 PriorityTier=3000 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=800 TotalNodes=20 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=8500 MaxMemPerCPU=9500
  TRES=cpu=800,mem=7500G,node=20,billing=800
```

## akya-cuda Kuyruğu

- Toplam CPU: 800
- Toplam Node: 20
- Maksimum süre: 3 gün ( MaxTime=3-00:00:00 )
- Bellek: 7.5 TB toplam ( mem=7500G )
- Kullanılan Node Aralığı: akya[1-20]
- GPU: 4×Nvidia V100 (16GB NVLink) her sunucuda
- DefMemPerCPU: 8500 MB (her CPU çekirdeği için varsayılan bellek)

```
(base) bash-5.1$ scontrol show partition=barbun-cuda
PartitionName=barbun-cuda
  AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
  AllocNodes=ALL Default=NO QoS=N/A
  DefaultTime=00:02:00 DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=3-00:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
  Nodes=barbun[121-144]
  PriorityJobFactor=3000 PriorityTier=3000 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=960 TotalNodes=24 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=8500 MaxMemPerCPU=9500
  TRES=cpu=960,mem=9000G,node=24,billing=960
```

## barbun-cuda Kuyruğu

- Toplam CPU: 960
- Toplam Node: 24
- Maksimum süre: 3 gün
- Bellek: 9 TB toplam ( mem=9000G )
- Kullanılan Node Aralığı: barbun[121-144]
- GPU: 2×Nvidia P100 (16GB) her sunucuda
- DefMemPerCPU: 8500 MB

```
● (base) bash-5.1$ scontrol show partition=debug
PartitionName=debug
  AllowGroups=ALL AllowAccounts=ALL AllowQos=ALL
  AllocNodes=ALL Default=NO QoS=debug
  DefaultTime=00:02:00 DisableRootJobs=NO ExclusiveUser=NO GraceTime=0 Hidden=NO
  MaxNodes=UNLIMITED MaxTime=04:00:00 MinNodes=0 LLN=NO MaxCPUsPerNode=UNLIMITED MaxCPUsPerSocket=UNLIMITED
  Nodes=akya[1-10],barbun[1-40,120-130],hamsi[1-40],orfoz[1-100]
  PriorityJobFactor=5000 PriorityTier=5000 RootOnly=NO ReqResv=NO OverSubscribe=NO
  OverTimeLimit=NONE PreemptMode=OFF
  State=UP TotalCPUs=15880 TotalNodes=201 SelectTypeParameters=NONE
  JobDefaults=(null)
  DefMemPerCPU=2000 MaxMemPerCPU=2170
  TRES=cpu=15600,mem=56686000M,node=201,billing=15600
```

## debug Kuyruğu Özeti

- **Amaç:** Kısa süreli test ve deneme işleri için kullanılır.
- **Maksimum süre:** 4 saat ( `MaxTime=04:00:00` )
- **Toplam CPU:** 15.880
- **Toplam Node:** 201
- **Kullanılan Node'lar:**
  - `akya[1-10]`
  - `barbun[1-40,120-130]`
  - `hamsi[1-40]`
  - `orfoz[1-100]`
- **DefMemPerCPU:** 2000 MB (her çekirdek için varsayılan bellek).
- **MaxMemPerCPU:** 2170 MB.
- **Toplam Bellek:**  $\approx$  56.8 TB ( `mem=56686000M` ).



# Dosya Sistemleri ve Depolama

Hesaplama kümelerinde kullanıcı verileri ve hesaplama çıktıları, **yüksek performanslı merkezi dosya sistemlerinde** yönetilir.

Bu sistemler **paralel dosya sistemi** altyapısı ile çalışır.

Üç ana dosya sistemi vardır:

- /arf/home → WEKA tabanlı ev dizini
- /arf/scratch → geçici (scratch) alanı
- /truba/home → LUSTRE tabanlı merkezi depolama alanı

| Dosya Sistemi             | Kullanım Amacı   | Kota                 | Performans                | Yaşam Süresi       |
|---------------------------|------------------|----------------------|---------------------------|--------------------|
| <code>/arf/home</code>    | Ev dizini        | 100 GB<br>100K dosya | Yüksek hız<br>Güvenilir   | Kullanıcı kontrolü |
| <code>/arf/scratch</code> | Geçici hesaplama | 1 TB<br>200K dosya   | Yüksek hız<br>Paralel I/O | En fazla 1 ay      |
| <code>/truba/home</code>  | Merkezi depolama | 2 TB<br>100K dosya   | Orta hız                  | Geçici depolama    |

## **ARF Ev Dizini (/arf/home/\$USER):**

- **Amaç:** Kullanıcıların kişisel dosyaları, betikleri ve küçük uygulama kurulumlarını saklaması.
- **Kullanım:**
  - Betik ve konfigürasyon dosyaları
  - Küçük uygulama kurulumları
  - Girdi/parametre dosyaları
  - Kalıcı olarak saklanacak küçük çıktılar
- **Kısıtlamalar:**
  - Büyük veri setleri saklanmamalı
  - Yoğun I/O işlemleri yapılmamalı
  - Uzun süreli arşivleme için kullanılmamalı

## ARF Scratch Dizini (/arf/scratch/\$USER):

- **Amaç:** Aktif hesaplama işleri için geçici, yüksek performanslı depolama alanı.
- **Kullanım:**
  - Hesaplama işlerinin çalıştırılması
  - Geçici çıktıların saklanması
  - Büyük veri setleriyle çalışma
  - Paralel I/O gerektiren uygulamalar
- **Uyarılar:**
  - Dosyalar **periyodik olarak silinir**
  - Önemli veriler mutlaka başka yere kopyalanmalıdır
  - Uzun süreli saklama yapılmamalıdır



- **TRUBA Merkezi Depolama (/truba/home/\$USER):**
  - **Amaç:** Projeler süresince geçici veri saklama alanı.
  - **Kullanım:**
    - Proje verilerinin geçici depolanması
    - Scratch çıktılarının transferi
    - Büyük veri setlerinin geçici saklanması

# Kota Aşım Çözümleri #

Kota limitine yaklaştığınızda veya aştığınızda:

## 1. Gereksiz dosyaları silin:

```
# Büyük dosyaları bul  
find /arf/home/$USER -type f -size +100M -ls  
  
# Eski dosyaları bul (30 günden eski)  
find /arf/scratch/$USER -type f -atime +30 -ls
```

## 2. Dosyaları arşivleyin:

```
# Sıkıştırılmış arşiv oluştur  
tar -czf arşiv.tar.gz klasor_adi/  
  
# Orijinal dosyaları sil  
rm -rf klasor_adi/
```

## 3. Verileri yerel bilgisayara indirin:

Veri transferi için [Dosya Transferi](#) bölümündeki yönergeleri takip edebilirsiniz.

# Dosya Sayısı (inode) Yönetimi

Dosya sayısı limiti, sistem performansını korumak için kritik önem taşır.

## Inode Optimizasyon Stratejileri

### 1. Merkezi Yazılımları Kullanın:

- `module load` sistemini kullanın
- Konteyner teknolojilerini tercih edin

### 2. Dosya Birleştirme:

```
# Küçük dosyaları birleştir
cat dosya1.txt dosya2.txt > birlesik_dosya.txt

# Çoklu dosyaları tek arşivde topla
tar -czf veri_seti.tar.gz *.dat
```

### 3. Anaconda/Conda Kullanımından Kaçının:

#### ⚠ Uyarı

`/arf` ve `/truba` dosya sistemlerine Anaconda, Miniconda, conda veya pip ile paket kurulumu yapılmamalıdır. Bu araçlar binlerce küçük dosya oluşturarak sistem performansını ciddi şekilde düşürür. Kullanım detayına [Python Kılavuzu](#) bölümünden ulaşabilirsiniz.

# Veri Yaşam Döngüsü Politikaları

## Ev Dizini (``/arf/home``):

- Kullanıcı kontrolünde yaşam süresi
- Düzenli temizlik önerilir
- Kritik veriler için yedekleme zorunlu

## Scratch Alanı (``/arf/scratch``):

- Maksimum 30 gün yaşam süresi
- Otomatik temizleme uygulanır
- Geçici dosyalar için tasarlanmıştır

## Merkezi Depolama (``/truba/home``):

- Proje süresi boyunca geçici depolama
- Uzun vadeli arşivleme için uygun değil

# En İyi Uygulamalar ve Öneriler

## Performans Optimizasyonu

### 1. Doğru Dosya Sistemi Seçimi:

- Hesaplama işleri için `/arf/scratch` kullanın
- Küçük dosyalar için `/arf/home` tercih edin
- Büyük veri setleri için `/truba/home` değerlendirin

### 2. Geçici Dosya Yönetimi:

```
# İş bitiminde geçici dosyaları temizle
export TMPDIR=/arf/scratch/$USER/tmp
mkdir -p $TMPDIR

# İş sonunda temizlik
trap 'rm -rf $TMPDIR' EXIT
```

Ben Scratch'te tutup çalıştırıyordum. Onları /arf/home'a taşıdım ve laptop'a indirdim.

### 1. Kota Aşım Hatası:

```
# Disk kullanımını kontrol et
du -sh /arf/home/$USER

# Büyük dosyaları bul
find /arf/home/$USER -type f -size +100M -exec ls -lh {} \;
```

41G /arf/home/oceylan

2.1G /arf/scratch/oceylan

### 2. İnode Limiti Aşımı:

```
# Dosya sayısını kontrol et
find /arf/home/$USER -type f | wc -l

# Küçük dosyaları birleştir veya sil
```

214953 /arf/home/oceylan

51012 /arf/scratch/oceylan

### 3. Erişim İzni Sorunları:

```
# Dosya izinlerini kontrol et
ls -la /arf/home/$USER

# Gerekğinde izinleri düzelt
chmod 755 /arf/home/$USER
```

```
Account: oceylan
Core-Hour Usage      : 71.0222 / 450000 hours
Value of Usage       :20.5964 TL
/truba Disk Quota    :35 GB / 2097 GB
/truba File Quota    :312147 / 200000
arf:/home/oceylan    :42.51 GB / 100.00 GB
arf:/scratch/oceylan :2.46 GB / 2 TB
```



# Diğer GPU Kümeleri

ARF ACC, büyük ölçekli araştırma projeleri ve ileri düzey hesaplama gereksinimleri için özel olarak yapılandırılmış GPU tabanlı bir HPC kümesidir.

Modern donanım altyapısına sahiptir ve yüksek performanslı hesaplamalar için tasarlanmıştır.

## Kullanım Politikaları

### **kolyoz-cuda ve palamut-cuda kümeleri:**

- Yalnızca T.C. Cumhurbaşkanlığı Strateji ve Bütçe Başkanlığı tarafından desteklenen altyapı projeleri
- veya TÜBİTAK ULAKBİM kapsamında sözleşmeli proje araştırmacıları tarafından kullanılabilir.

### **barbun-cuda ve akya-cuda kümeleri:**

- Tüm kullanıcıların erişimine açık.

# Yüksek Başarımli Hesaplama

- YBH sistemlerinde işler doğrudan sunucuda değil, bir kuyruk (queue) sistemi üzerinden yürütölür.
- Hesaplama/analiz işleri, SLURM iş betiđi hazırlanarak kuyruk yöneticisine (sbatch job.slurm) gönderilir.
- Kuyruk sistemi, birden fazla kullanıcının kaynak taleplerini adil ve verimli biçimde yönetir.
- İşler, mevcut kaynaklara (CPU, GPU, bellek) ve önceliklere göre sıralı şekilde çalıştırılır.
- Bu yapı sayesinde kaynak çakışmaları ve aşırı yüklenme önlenir.



Kuyruklar (partitions) #

| Kuyruk       | Sunucu                                              | Adet | Maks. Süre | Min. Çekirdek | DefMemPerCore            | MaxMemPerCore            | Durum |
|--------------|-----------------------------------------------------|------|------------|---------------|--------------------------|--------------------------|-------|
| debug        | orfoz, hamsi,<br>barbun, barbun-<br>cuda, akya-cuda | 200  | 4 saat     | 1             | Kuyruğa göre<br>değişir. | Kuyruğa göre<br>değişir. | Aktif |
| smp          | orkinos                                             | 1    | 3 gün      | 4             | 17000MB                  | 18400MB                  | Özel  |
| barbun       | barbun                                              | 119  | 3 gün      | 20            | 8500MB                   | 9500MB                   | Aktif |
| barbun-cuda  | barbun-cuda                                         | 24   | 3 gün      | 20            | 8500MB                   | 9500MB                   | Aktif |
| akya-cuda    | akya-cuda                                           | 24   | 3 gün      | 10            | 8500MB                   | 9500MB                   | Aktif |
| hamsi        | hamsi                                               | 144  | 3 gün      | 28            | 3400MB                   | 3400MB                   | Aktif |
| orfoz        | orfoz                                               | 504  | 3 gün      | 56            | 2000MB                   | 2295MB                   | Aktif |
| palamut-cuda | palamut                                             | 9    | 3 gün      | 16            | 7500MB                   | 16384MB                  | Aktif |
| kolyoz-cuda  | kolyoz1-kolyoz24                                    | 24   | 3 gün      | 16            | 16GB                     | 16GB                     | Aktif |
| kolyoz-cuda  | kolyoz25-kolyoz72                                   | 24   | 3 gün      | 16            | 14GB                     | 16GB                     | Aktif |



```
● (base) bash-5.1$ sinfo
PARTITION  AVAIL  TIMELIMIT  NODES  STATE NODELIST
orfoz*      up 3-00:00:00    23   down orfoz[19,22,31,54,84,123,173,188,197-198,224,229,231,246,256,305,369,385-390]
orfoz*      up 3-00:00:00    16   mix  orfoz[32,60,83,108,115,135,164,168,258,274,307,380-384]
orfoz*      up 3-00:00:00   285  alloc orfoz[1-13,15-18,20-21,23-30,33-36,38,41-53,55-59,61-64,66-69,77-82,85-91,94-107,109,111-114,116-122,
124-134,136-139,141-159,161-163,165-167,169-172,174-187,189-196,219-223,225-228,230,232-245,247-255,257,259-273,275-276,278,280-293,295-302,306,
308-336,343-344,370-379,391-396]
orfoz*      up 3-00:00:00    72   idle orfoz[14,37,39-40,65,70-76,92-93,110,140,160,199-218,277,279,294,303-304,337-342,345-368]
debug       up   4:00:00     1  drain* barbun1
debug       up   4:00:00     5   down orfoz[19,22,31,54,84]
debug       up   4:00:00     4   mix  barbun123,orfoz[32,60,83]
debug       up   4:00:00   107  alloc akya[1-10],barbun[120-122,124-130],hamsi[13-17,19,23-24,28],orfoz[1-13,15-18,20-21,23-30,33-36,38,41-
53,55-59,61-64,66-69,77-82,85-91,94-100]
debug       up   4:00:00    84   idle barbun[2-40],hamsi[1-12,18,20-22,25-27,29-40],orfoz[14,37,39-40,65,70-76,92-93]
barbun       up 3-00:00:00     3  drain* barbun[1,109,111]
barbun       up 3-00:00:00     1  down* barbun119
barbun       up 3-00:00:00     1  drain barbun101
barbun       up 3-00:00:00     4   mix  barbun[105-106,114-115]
barbun       up 3-00:00:00    28  alloc barbun[41-45,48-52,88-90,95,99-100,102-104,107-108,110,112-113,116-118,120]
barbun       up 3-00:00:00    83   idle barbun[2-40,46-47,53-87,91-94,96-98]
hamsi        up 3-00:00:00     1  down* hamsi95
hamsi        up 3-00:00:00     1   mix  hamsi93
hamsi        up 3-00:00:00    58  alloc hamsi[13-17,19,23-24,28,41-43,48-54,58,60-62,67-73,75-82,94,96-97,101-114,125,135-136]
hamsi        up 3-00:00:00    84   idle hamsi[1-12,18,20-22,25-27,29-40,44-47,55-57,59,63-66,74,83-92,98-100,115-124,126-134,137-144]
smp          up 3-00:00:00     1   idle orkinos1
akya-cuda    up 3-00:00:00     1  drain* akya17
akya-cuda    up 3-00:00:00     5  drain akya[15-16,18-20]
akya-cuda    up 3-00:00:00    14  alloc akya[1-14]
barbun-cuda  up 3-00:00:00     1   mix  barbun123
barbun-cuda  up 3-00:00:00    23  alloc barbun[121-122,124-144]
```

```
akya-cuda      up 3-00:00:00      1 drain* akya17
akya-cuda      up 3-00:00:00      5  drain akya[15-16,18-20]
akya-cuda      up 3-00:00:00     14  alloc akya[1-14]
```

## **akya-cuda**

```
akya-cuda      up 3-00:00:00      1 drain* akya17
akya-cuda      up 3-00:00:00      5  drain akya[15-16,18-20]
akya-cuda      up 3-00:00:00     14  alloc akya[1-14]
```

14 node tamamen dolu (alloc)

6 node bakımda veya arızalı (drain)

Yani şu anda akya-cuda'da boş (idle) node yok



```
barbun-cuda    up 3-00:00:00    1    mix barbun123  
barbun-cuda    up 3-00:00:00    23   alloc barbun[121-122,124-144]
```

## barbun-cuda

barbun-cuda up 3-00:00:00 1 mix barbun123

barbun-cuda up 3-00:00:00 23 alloc barbun[121-122,124-144]

23 node tamamen dolu (alloc)

1 node kısmen dolu

# Open OnDemand Arayüzü

← → ↻ Not Secure https://openondemand.yonetim/pun/sys/dashboard/activejobs

Dashboard Inbox - enes.cagla... ChatGPT (7) Feed | LinkedIn

Open OnDemand Files ▾ Jobs ▾ ARF ▾ SHELL ▾

Relaunch to update

Your Jobs ▾ All Clusters ▾

## Active Jobs

Show 50 ▾ entries Filter:

| ID | Name    | User                             | Account | Time Used | Queue    | Status      | Cluster   | Actions |
|----|---------|----------------------------------|---------|-----------|----------|-------------|-----------|---------|
| >  | 3169456 | akya_min                         | oceylan | oceylan   | 00:00:02 | akya-cuda   | Completed | arf     |
| >  | 3169052 | akya_max                         | oceylan | oceylan   | 00:00:00 | akya-cuda   | Queued    | arf     |
| >  | 3169458 | akya_max                         | oceylan | oceylan   | 00:00:00 | akya-cuda   | Queued    | arf     |
| >  | 3169457 | akya_ortalama                    | oceylan | oceylan   | 00:00:00 | akya-cuda   | Queued    | arf     |
| >  | 3169462 | barbun_max                       | oceylan | oceylan   | 00:00:01 | barbun-cuda | Completed | arf     |
| >  | 3169460 | barbun_min                       | oceylan | oceylan   | 00:00:02 | barbun-cuda | Completed | arf     |
| >  | 3169448 | sys/dashboard/sys/bc_jupyter_gpu | oceylan | oceylan   | 00:00:00 | debug       | Queued    | arf     |

Showing 1 to 7 of 7 entries

Previous 1 Next

<https://openondemand.yonetim>

# Grafana

Grafana, zaman serisi veri görselleştirme ve izleme için açık kaynaklı bir platformdur. Genellikle sistem metrikleri, uygulama günlükleri ve altyapı verileri gibi gerçek zamanlı verileri takip etmek amacıyla kullanılır.



<http://grafana.yonetim:3000/login>



# İnteraktif İşler İçin Debug

- Debug kuyruğu, normal kuyruklardan farklı olarak çok kısa sürede sonuç almak isteyen kullanıcılar için tasarlanmıştır.
- Bu kuyruk, kod testleri, küçük veri denemeleri veya SLURM betiği kontrolleri yapmak için idealdir.

```
srun -p debug -N 1 -n 1 -c 110 -A kullanıcı_adi -J test --time=0:30:00 --pty /usr/bin/bash -i
```

-N 1 → 1 node ister.

-n 1 → 1 görev çalıştırır.

-c 110 → 110 CPU çekirdeği talep eder (örnek değer).

-A kullanıcı\_adi → Kullanıcı hesabı (örnek: -A oceylan).

-J test → İşin adı (örnek: test).

--time=0:30:00 → Maksimum çalışma süresi 30 dakikadır.

--pty /usr/bin/bash -i → Etkileşimli bash oturumu açar.



```
run -p debug -C barbun-cuda -N 1 -n 1 -c 20 --gres=gpu:1 -A kullanıcı_adi -J test --time=0:30:00 --pty /usr/bin/bash -i
```

```
srun -p debug -C barbun-cuda -N 1 -n 1 -c 20 --gres=gpu:1 -A oceylan -J barbun_test --time=0:30:00  
--pty /usr/bin/bash -i
```

```
srun -p debug -C akya-cuda -N 1 -n 1 -c 10 --gres=gpu:1 -A kullanıcı_adi -J test --time=0:30:00 --pty /usr/bin/bash -i
```

```
srun -p debug -C akya-cuda -N 1 -n 1 -c 10 --gres=gpu:1 -A oceylan -J akya_test --time=0:30:00 --pty  
/usr/bin/bash -i
```

# Kuyruk Bilgisi

```
(base) bash-5.1$ lssrv
```

| TRUBA Parititons State           |          |         |             |            |         |               |            |            |  |
|----------------------------------|----------|---------|-------------|------------|---------|---------------|------------|------------|--|
| PARTITION                        | CPUS     | CPUS    | WAIT. JOBS  | WAIT. JOBS | NODES   | MAX. JOB TIME | MIN. NODES | MAX. NODES |  |
| CORE                             | RAM (MB) |         |             |            |         |               |            |            |  |
| NAME                             | (FREE)   | (TOTAL) | (RESOURCES) | (TOTAL)    | (TOTAL) | (D-HH:MM:SS)  | PER JOB    | PER JOB    |  |
| PER NODE                         | PER CORE |         |             |            |         |               |            |            |  |
| akya-cuda                        | 0        | 800     | 1           | 135        | 20      | 3-00:00:00    | 1          | infinite   |  |
| 40                               | 9600     |         |             |            |         |               |            |            |  |
| barbun                           | 2920     | 4800    | 0           | 6          | 120     | 3-00:00:00    | 1          | infinite   |  |
| 40                               | 9600     |         |             |            |         |               |            |            |  |
| barbun-cuda                      | 140      | 960     | 1           | 15         | 24      | 3-00:00:00    | 1          | infinite   |  |
| 40                               | 9600     |         |             |            |         |               |            |            |  |
| debug                            | 4518     | 15880   | 1           | 9          | 201     | 4:00:00       | 1          | infinite   |  |
| 40+                              | 4788     |         |             |            |         |               |            |            |  |
| hamsi                            | 5362     | 8064    | 0           | 51         | 144     | 3-00:00:00    | 1          | infinite   |  |
| 56                               | 3420     |         |             |            |         |               |            |            |  |
| orfoz                            | 6934     | 44352   | 0           | 195        | 396     | 3-00:00:00    | 1          | infinite   |  |
| 112                              | 2285     |         |             |            |         |               |            |            |  |
| smp                              | 224      | 224     | 0           | 0          | 1       | 3-00:00:00    | 1          | infinite   |  |
| 224                              | 18428    |         |             |            |         |               |            |            |  |
| LAST UPDATE: 03 NOV 25 18:01 +03 |          |         |             |            |         |               |            |            |  |

## akya-cuda

800 CPU var, şu anda boş CPU = 0 → tamamen dolu (muhtemelen GPU işleri çalışıyor).  
135 job beklemede.

## barbun-cuda

960 CPU var, boş 140 → birkaç node dolu, bazıları boş.  
15 job beklemede.

## debug

15880 CPU var, boş 4518 → test işleri için yer var.  
Maksimum süre 4 saat.

# Modül Arama Aracı

- Avcı (avci) komutu, TRUBA'da mevcut yazılım modüllerini kolayca aramak için kullanılır.
- Yani module avail komutunun daha gelişmiş, filtrelenebilir ve renklendirilmiş versiyonudur.

## Örnekler

### 1. Basit Arama:

```
./avci gcc
```

`gcc` içeren tüm modülleri arar. Çıktı renklidir, yüklü modüller yeşil renkte gösterilir.

### 2. Tam Eşleşme:

```
./avci -exact openmpi
```

Sadece adı tam olarak `openmpi` olan modülleri listeler (sürüm dahil değildir).

### 3. Renkli Çıktıyı Devre Dışı Bırakma:

```
./avci -no-color cuda
```

Renk kullanılmadan, `cuda` içeren modüller listelenir.

### 4. Yardım Menüsü:

```
./avci -help
```

Kullanım ve opsiyonlar hakkında bilgi verir.

# SLURM Komutları

- `sbatch`
  - İş betiğini kuyruk sistemine gönderir.
  - Parametreler betik içinde veya doğrudan komutla verilebilir.
- `squeue`
  - Kullanıcının **bekleyen ve çalışan** işlerini listeler.
  - `squeue -u <kullanıcı_adı>` → sadece kendi işlerini gösterir.
- `sinfo`
  - Kuyruklardaki **mevcut sistem durumu** (boş/dolu/arıza) bilgilerini gösterir.
  - En hızlı başlayacak kuyruğu seçmek için kullanılır.
- `srun`
  - Küçük veya kısa süreli işleri **doğrudan çalıştırmak** için kullanılır.
  - `sbatch` 'in interaktif alternatifi.
- `scancel`
  - Kuyruktaki veya çalışan işleri **iptal eder**.
  - `scancel <JOBID>` şeklinde kullanılır.
- `sacct`
  - Tamamlanan veya geçmişte çalışmış işlerin **detaylı raporunu** verir.
  - Bellek kullanımı, süre, çalıştığı düğüm gibi bilgiler içerir.
- `sstat`
  - Çalışmakta olan işin **gerçek zamanlı kaynak kullanımını** (CPU, bellek vb.) gösterir.

# Slurm Betik Özellikleri

## • Temel Yapı

- Her SLURM betiği 3 ana bölümden oluşur:
  - 1 Başlangıç (parametreler)
  - 2 Gövde (modüller ve ayarlar)
  - 3 İş başlatma ve bitiş (çalıştırma komutları)
- Betik `#!/bin/bash` satırıyla başlar (yorumlayıcı tanımı).



## Başlangıç Bölümü – Parametreler

- Tüm tanımlar `#SBATCH` ile başlar.
- Temel örnek:

bash

```
#SBATCH -p <kuyruk_adi>           # Kuyruk adı
#SBATCH -A <hesap_adi>            # Kullanıcı hesabı
#SBATCH -J <is_adi>               # İş ismi
#SBATCH -N <node_sayısı>          # Node sayısı
#SBATCH -n <görev_sayısı>         # MPI görev sayısı
#SBATCH -c <çekirdek_sayısı>      # Görev başına çekirdek
#SBATCH --time=1:00:00            # Maksimum süre
#SBATCH --mem=16G                 # Bellek limiti
#SBATCH --gres=gpu:1              # GPU talebi
```

## | Gövde Bölümü – Modüller ve Ayarlar

- Gerekli kütüphaneler ve çevre değişkenleri yüklenir.
- Standart başlangıç örneği:

```
bash
```

```
export OMP_NUM_THREADS=1
```

```
module purge
```

```
module load comp/oneapi/2023
```

```
module load apps/lammps/29Aug2024_stable_oneapi-2024
```

- Bu bölüm, **hazırlık ve ortam kurulumunun** yapıldığı kısımdır.

## İşin Başlaması ve Bitişi

- Asıl hesaplama veya uygulama bu kısımda çağrılır.
- MPI işler** için `mpirun` kullanılır:

```
bash
```

```
mpirun <uygulama_adı>
```

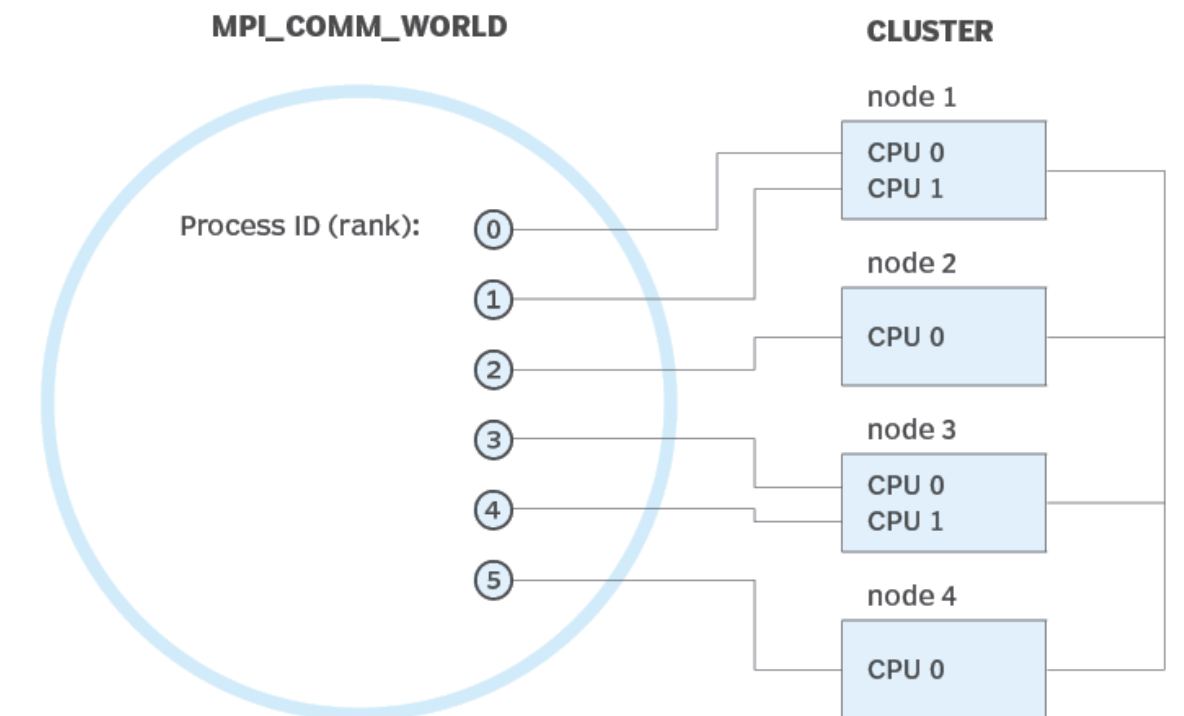
```
exit
```

- Çalışma sonunda kaynak kullanımı görmek için:

```
bash
```

```
sstat -j $SLURM_JOB_ID
```

## Message passing interface (MPI)



# Yazılımlar ve Uygulamalar

- TRUBA’da yaygın yazılımlar, kütüphaneler ve uygulamalar merkezi olarak kuruludur.
- Kullanıcıların öncelikle bu merkezi kurulumları kullanması önerilir.
- Merkezi yazılımlar, modül sistemi (Environment Modules) üzerinden yönetilir.

| Komut                                  | Açıklama                                 |  |
|----------------------------------------|------------------------------------------|--|
| <code>module available</code>          | Sistemde yüklü tüm yazılımları listeler. |  |
| <code>module load &lt;modül&gt;</code> | Belirtilen yazılımı ortamınıza yükler.   |  |

- Eğer aradığınız yazılım mevcut değilse veya sürümü uygun değilse, **kendi ev dizininize ( /arf/home/\$USER )** kurabilirsiniz.
- Kurulum sırasında TRUBA’nın mevcut **güncel derleyici (compiler)** ve **kütüphanelerini** kullanmak tavsiye edilir.

## Merkezi Yazılım ve Veri Dizinleri

| Tür                   | Dizin               | Açıklama                                       |  |
|-----------------------|---------------------|------------------------------------------------|-------------------------------------------------------------------------------------|
| Uygulamalar           | /arf/sw/apps        | Merkezi kurulu yazılım uygulamaları            |                                                                                     |
| Kütüphaneler          | /arf/sw/lib         | Ortak kullanılan kütüphaneler                  |                                                                                     |
| Konteynerler          | /arf/sw/containers  | Singularity/Docker benzeri konteyner dosyaları |                                                                                     |
| Derleyiciler          | /arf/sw/comp        | C, C++, Fortran vb. derleyiciler               |                                                                                     |
| Modüller              | /arf/sw/modulefiles | Modül sisteminin yapılandırma dosyaları        |                                                                                     |
| Kaynak Kodlar         | /arf/sw/src         | Yazılım kaynak kodlarının bulunduğu dizin      |                                                                                     |
| Veri Setleri          | /arf/repo           | Ortak kullanılabilen veri setleri              |                                                                                     |
| Örnek SLURM Betikleri | /arf/sw/scripts     | Hazır örnek iş gönderim dosyaları              |                                                                                     |



```
#!/bin/bash
#SBATCH -p akya-cuda          # Kuyruk adi: Uzerinde GPU olan kuyruk olmasina dikkat edin.
#SBATCH -A [USERNAME]        # Kullanici adi
#SBATCH -J print_gpu         # Gonderilen isin ismi
#SBATCH -o print_gpu.out     # Ciktinin yazilacagi dosya adi
#SBATCH --gres=gpu:1         # Her bir sunucuda kac GPU istiyorsunuz? Kumeleri kontrol edin.
#SBATCH -N 1                 # Gorev kac node'da calisacak?
#SBATCH -n 1                 # Ayni gorevden kac adet calistirilacak?
#SBATCH --cpus-per-task 10   # Her bir gorev kac cekirdek kullanacak? Kumeleri kontrol edin.
#SBATCH --time=1:00:00      # Sure siniri koyun.

# Modüller
# Çalıştırılacak komutlar
```

### CPU–GPU Oranı Kuralları (Küme Bazlı)

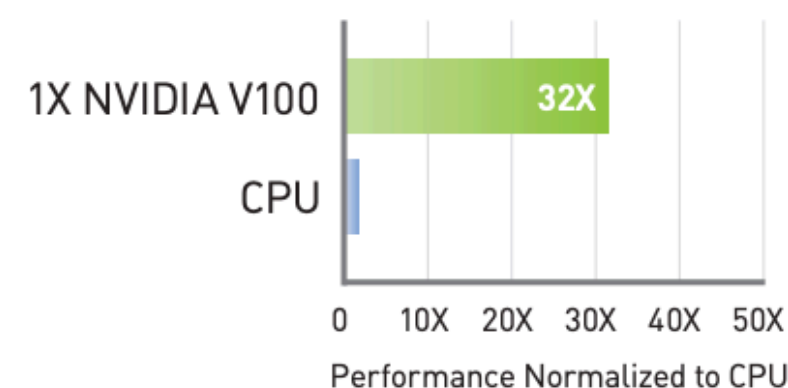
| Küme        | GPU Modeli | Gerekli CPU Sayısı |
|-------------|------------|--------------------|
| barbun-cuda | 2×P100     | 20 × GPU_SAYISI    |
| akya-cuda   | 4×V100     | 10 × GPU_SAYISI    |

# SPECIFICATIONS

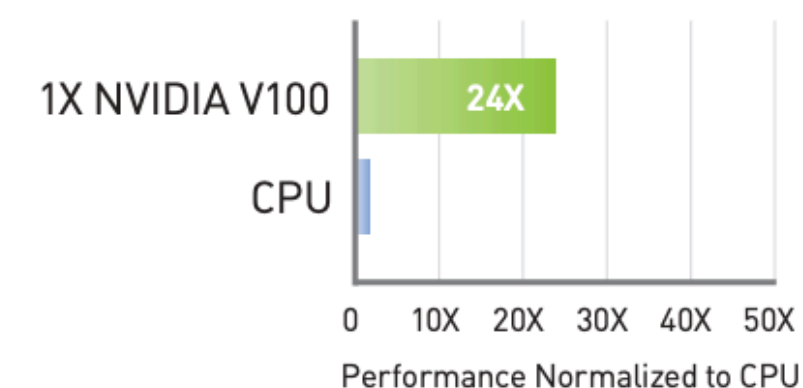
# NVIDIA Tesla V100

|                              | V100<br>PCIe                           | V100<br>SXM2   | V100S<br>PCIe           |
|------------------------------|----------------------------------------|----------------|-------------------------|
| GPU Architecture             | NVIDIA Volta                           |                |                         |
| NVIDIA Tensor Cores          | 640                                    |                |                         |
| NVIDIA CUDA® Cores           | 5,120                                  |                |                         |
| Double-Precision Performance | 7 TFLOPS                               | 7.8 TFLOPS     | 8.2 TFLOPS              |
| Single-Precision Performance | 14 TFLOPS                              | 15.7 TFLOPS    | 16.4 TFLOPS             |
| Tensor Performance           | 112 TFLOPS                             | 125 TFLOPS     | 130 TFLOPS              |
| GPU Memory                   | 32 GB /16 GB HBM2                      |                | 32 GB HBM2              |
| Memory Bandwidth             | 900 GB/sec                             |                | 1134 GB/sec             |
| ECC                          | Yes                                    |                |                         |
| Interconnect Bandwidth       | 32 GB/sec                              | 300 GB/sec     | 32 GB/sec               |
| System Interface             | PCIe Gen3                              | NVIDIA NVLink™ | PCIe Gen3               |
| Form Factor                  | PCIe Full Height/Length                | SXM2           | PCIe Full Height/Length |
| Max Power Consumption        | 250 W                                  | 300 W          | 250 W                   |
| Thermal Solution             | Passive                                |                |                         |
| Compute APIs                 | CUDA, DirectCompute, OpenCL™, OpenACC® |                |                         |

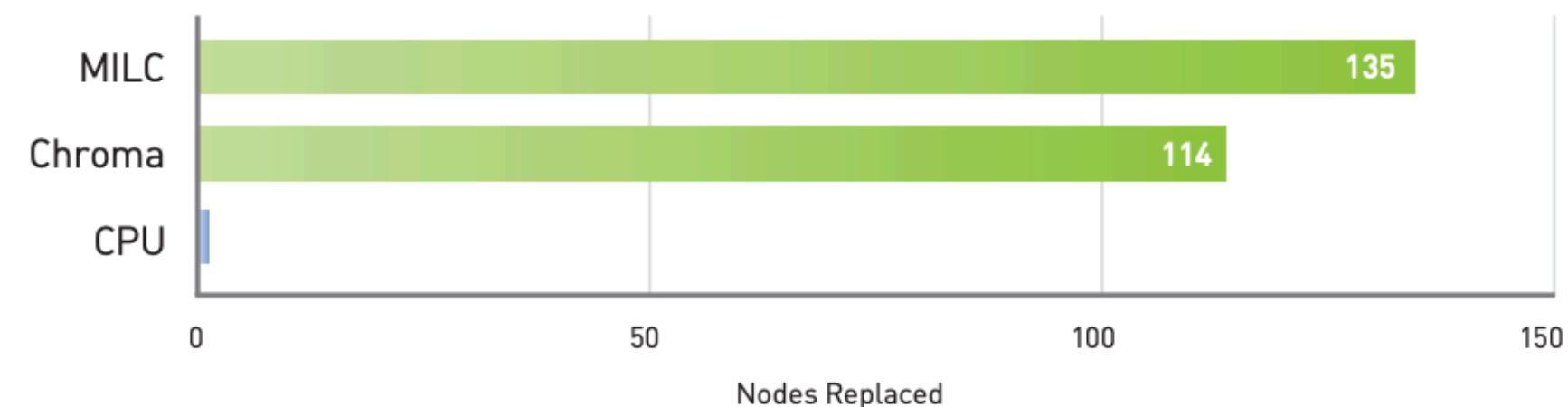
32X Faster Training Throughput than a CPU<sup>1</sup>



24X Higher Inference Throughput than a CPU Server<sup>2</sup>



HPC: One V100 Server Node Replaces Up to 135 CPU-Only Server Nodes<sup>3</sup>



# NVIDIA Tesla P100

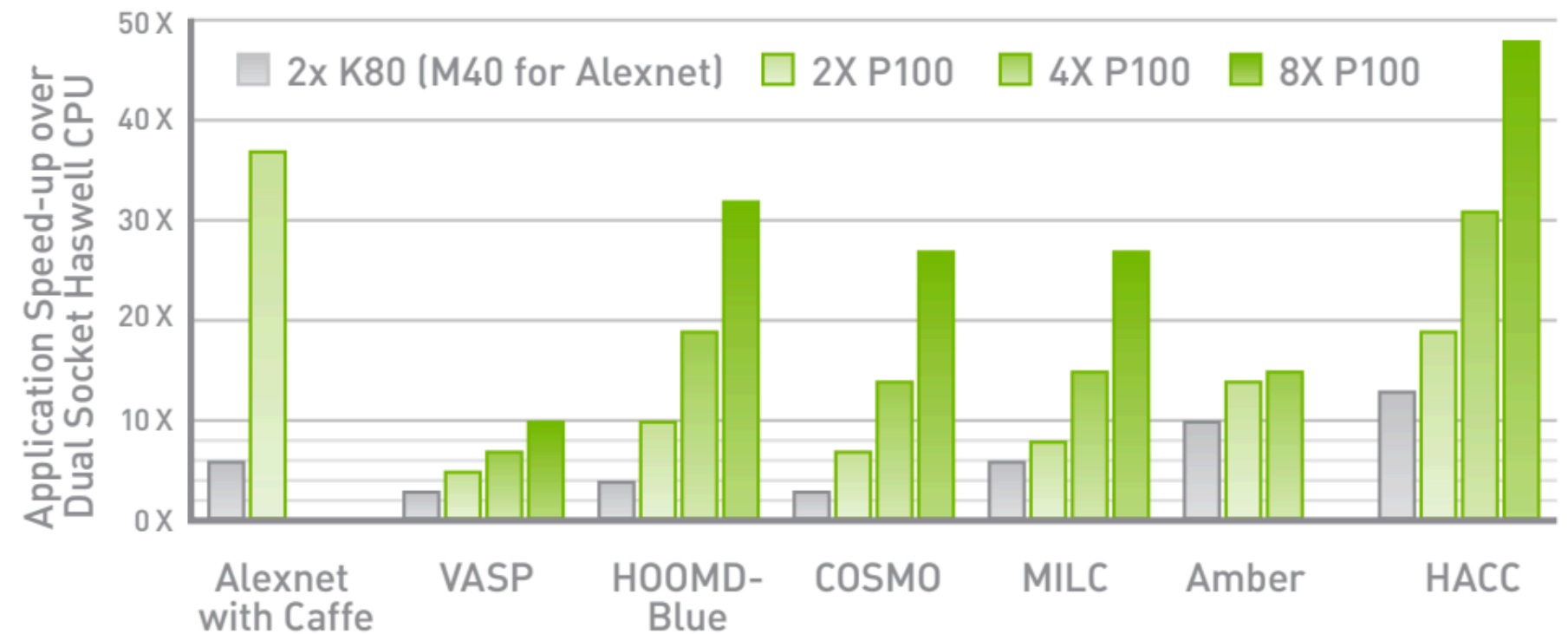
## SPECIFICATIONS

|                              |                                                                |
|------------------------------|----------------------------------------------------------------|
| GPU Architecture             | <b>NVIDIA Pascal</b>                                           |
| NVIDIA CUDA® Cores           | <b>3584</b>                                                    |
| Double-Precision Performance | <b>5.3 TeraFLOPS</b>                                           |
| Single-Precision Performance | <b>10.6 TeraFLOPS</b>                                          |
| Half-Precision Performance   | <b>21.2 TeraFLOPS</b>                                          |
| GPU Memory                   | <b>16 GB CoWoS HBM2</b>                                        |
| Memory Bandwidth             | <b>732 GB/s</b>                                                |
| Interconnect                 | <b>NVIDIA NVLink</b>                                           |
| Max Power Consumption        | <b>300 W</b>                                                   |
| ECC                          | <b>Native support with no capacity or performance overhead</b> |
| Thermal Solution             | <b>Passive</b>                                                 |
| Form Factor                  | <b>SXM2</b>                                                    |
| Compute APIs                 | <b>NVIDIA CUDA, DirectCompute, OpenCL™, OpenACC</b>            |

TeraFLOPS measurements with NVIDIA GPU Boost™ technology

## DATA CENTER APPLICATIONS

### NVIDIA Tesla P100 Performance



CPU: 16 cores, E5-2698v3 @ 2.30GHz. 256GB System Memory. Tesla K80 GPUs: 2x Dual GPU K80s, Pre-Production Tesla P100

# SPECIFICATIONS

|                              | V100<br>PCIe                           | V100<br>SXM2   | V100S<br>PCIe           |
|------------------------------|----------------------------------------|----------------|-------------------------|
| GPU Architecture             | NVIDIA Volta                           |                |                         |
| NVIDIA Tensor Cores          | 640                                    |                |                         |
| NVIDIA CUDA® Cores           | 5,120                                  |                |                         |
| Double-Precision Performance | 7 TFLOPS                               | 7.8 TFLOPS     | 8.2 TFLOPS              |
| Single-Precision Performance | 14 TFLOPS                              | 15.7 TFLOPS    | 16.4 TFLOPS             |
| Tensor Performance           | 112 TFLOPS                             | 125 TFLOPS     | 130 TFLOPS              |
| GPU Memory                   | 32 GB /16 GB HBM2                      |                | 32 GB HBM2              |
| Memory Bandwidth             | 900 GB/sec                             |                | 1134 GB/sec             |
| ECC                          | Yes                                    |                |                         |
| Interconnect Bandwidth       | 32 GB/sec                              | 300 GB/sec     | 32 GB/sec               |
| System Interface             | PCIe Gen3                              | NVIDIA NVLink™ | PCIe Gen3               |
| Form Factor                  | PCIe Full Height/Length                | SXM2           | PCIe Full Height/Length |
| Max Power Consumption        | 250 W                                  | 300 W          | 250 W                   |
| Thermal Solution             | Passive                                |                |                         |
| Compute APIs                 | CUDA, DirectCompute, OpenCL™, OpenACC® |                |                         |

# SPECIFICATIONS

|                              |                                                         |
|------------------------------|---------------------------------------------------------|
| GPU Architecture             | NVIDIA Pascal                                           |
| NVIDIA CUDA® Cores           | 3584                                                    |
| Double-Precision Performance | 5.3 TeraFLOPS                                           |
| Single-Precision Performance | 10.6 TeraFLOPS                                          |
| Half-Precision Performance   | 21.2 TeraFLOPS                                          |
| GPU Memory                   | 16 GB CoWoS HBM2                                        |
| Memory Bandwidth             | 732 GB/s                                                |
| Interconnect                 | NVIDIA NVLink                                           |
| Max Power Consumption        | 300 W                                                   |
| ECC                          | Native support with no capacity or performance overhead |
| Thermal Solution             | Passive                                                 |
| Form Factor                  | SXM2                                                    |
| Compute APIs                 | NVIDIA CUDA, DirectCompute, OpenCL™, OpenACC            |

TeraFLOPS measurements with NVIDIA GPU Boost™ technology

| Feature                    | Tesla V100       | Tesla P100       |
|----------------------------|------------------|------------------|
| Architecture               | Volta            | Pascal           |
| Code name                  | GV100            | GP100            |
| Release Year               | 2017             | 2016             |
| Cores / GPU                | 5120             | 3584             |
| GPU Boost Clock            | 1530 MHz         | 1480 MHz         |
| Tensor Cores / GPU         | 640              | NA               |
| Memory type                | HBM2             | HBM2             |
| Maximum RAM amount         | 32 GB            | 16 GB            |
| Memory clock speed         | 1758 MHz         | 1430 MHz         |
| Memory bandwidth           | 900.1 GB/s       | 720.9 GB/s       |
| CUDA Support               | From 7.0 Version | From 6.0 Version |
| Floating-point performance | 14,029 gflops    | 10,609 gflops    |



# Akya Cuda Kullanımı

```
module purge
module load apps/truba-ai/gpu-2024.0
```

```
srun -p akya-cuda -A oceylan -N 1 -n 1 -c 10 --gres=gpu:1 --time=00:45:00 --pty bash -i
```

```
bash-5.1$ srun -p akya-cuda -A oceylan -N 1 -n 1 -c 10 --gres=gpu:1 --time=00:15:00 --pty bash -i
srun: Interaktif isler sadece debug kuyrugunda calistirilabilir
srun: Interaktif isler sadece debug kuyrugunda calistirilabilir
srun: job 3170645 queued and waiting for resources
srun: job 3170645 has been allocated resources
bash-5.1$ ls
mobileNetGPU  mobilenet_test
bash-5.1$ hostname
nvidia-smi
akya13
Mon Nov  3 23:20:04 2025
```

|                      |                      |      |               |                           |              |          |             |                    |  |  |     |
|----------------------|----------------------|------|---------------|---------------------------|--------------|----------|-------------|--------------------|--|--|-----|
| NVIDIA-SMI 565.57.01 |                      |      |               | Driver Version: 565.57.01 |              |          |             | CUDA Version: 12.7 |  |  |     |
| GPU                  | Name                 | Perf | Persistence-M | Bus-Id                    | Disp.A       | Volatile | Uncorr. ECC |                    |  |  |     |
| Fan                  | Temp                 |      | Pwr:Usage/Cap |                           | Memory-Usage | GPU-Util | Compute M.  |                    |  |  |     |
|                      |                      |      |               |                           |              |          | MIG M.      |                    |  |  |     |
| 0                    | Tesla V100-SXM2-16GB |      | On            | 00000000:61:00.0          | Off          |          | 0           |                    |  |  |     |
| N/A                  | 29C                  | P0   | 38W / 300W    | 1MiB / 16384MiB           |              | 0%       | Default     |                    |  |  | N/A |

```
No running processes found
```

# Barbun Cuda Kullanımı

```
module purge  
module load apps/truba-ai/gpu-2024.0
```

```
srun -p barbun-cuda -A oceylan -N 1 -n 1 -c 20 --gres=gpu:1 --  
time=00:15:00 --pty bash -i
```

Bağlanamadım çok sıra vardı ertesi güne veriyordu tahmini bağlanmayı

# Pretrained MobileNet GPU Run with TensorFlow on Python

cannot find tensorflow  
solution  
module purge  
module load apps/truba-ai/gpu-2024.0

Akya Cuda’ya bağlanıp python mobilenet\_gpu\_test.py yazınca çıkan sonuçlar

| 🇹🇷 === Batch Size GPU Benchmark Results === |              |                |                    |                     |                      |               |               |  |  |  |  |  |  |
|---------------------------------------------|--------------|----------------|--------------------|---------------------|----------------------|---------------|---------------|--|--|--|--|--|--|
| Batch Size                                  | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/sec) | Top-1 Acc (%) | Top-5 Acc (%) |  |  |  |  |  |  |
| 16                                          | 50000        | 231.07         | 0.0739             | 4.62                | 216.4                | 69.06         | 88.52         |  |  |  |  |  |  |
| 32                                          | 50000        | 133.74         | 0.0856             | 2.67                | 373.9                | 69.06         | 88.52         |  |  |  |  |  |  |
| 64                                          | 50000        | 85.32          | 0.1091             | 1.71                | 586.0                | 69.06         | 88.52         |  |  |  |  |  |  |
| 128                                         | 50000        | 53.16          | 0.1359             | 1.06                | 940.6                | 69.06         | 88.52         |  |  |  |  |  |  |
| 256                                         | 50000        | 37.66          | 0.1921             | 0.75                | 1327.7               | 69.06         | 88.52         |  |  |  |  |  |  |
| 512                                         | 50000        | 29.81          | 0.3042             | 0.60                | 1677.1               | 69.06         | 88.52         |  |  |  |  |  |  |
| 1024                                        | 50000        | 26.54          | 0.5415             | 0.53                | 1884.3               | 69.06         | 88.52         |  |  |  |  |  |  |
| 2048                                        | 50000        | 24.95          | 0.9979             | 0.50                | 2004.3               | 69.06         | 88.52         |  |  |  |  |  |  |
| 4096                                        | 50000        | 25.31          | 1.9471             | 0.51                | 1975.3               | 69.06         | 88.52         |  |  |  |  |  |  |

# Pretrained MobileNet GPU Run with PyTorch on Python

Akya Cuda'ya bağlanıp python mobilenet\_gpu\_pytorch.py yazınca çıkan sonuçlar

| 🇹🇷 === PyTorch GPU Benchmark Results === |              |                |                    |                     |                    |               |               |  |  |  |  |
|------------------------------------------|--------------|----------------|--------------------|---------------------|--------------------|---------------|---------------|--|--|--|--|
| Batch Size                               | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/s) | Top-1 Acc (%) | Top-5 Acc (%) |  |  |  |  |
| 16                                       | 50000        | 65.18          | 0.0209             | 1.30                | 767.1              | 69.25         | 88.81         |  |  |  |  |
| 32                                       | 50000        | 60.40          | 0.0386             | 1.21                | 827.9              | 69.25         | 88.81         |  |  |  |  |
| 64                                       | 50000        | 59.68          | 0.0763             | 1.19                | 837.8              | 69.25         | 88.81         |  |  |  |  |
| 128                                      | 50000        | 58.17          | 0.1488             | 1.16                | 859.6              | 69.25         | 88.81         |  |  |  |  |
| 256                                      | 50000        | 58.27          | 0.2973             | 1.17                | 858.0              | 69.25         | 88.81         |  |  |  |  |
| 512                                      | 50000        | 59.37          | 0.6058             | 1.19                | 842.2              | 69.25         | 88.81         |  |  |  |  |
| 1024                                     | 50000        | 61.97          | 1.2647             | 1.24                | 806.9              | 69.25         | 88.81         |  |  |  |  |

Batch 2048: 0%|

| 0/25 [00:20<?, ?batch/s]

Traceback (most recent call last):

File "/arf/scratch/oceylan/mobileNetGPU/mobilenet\_gpu\_pytorch.py", line 69, in  
<module>

torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate 9.19 GiB. GPU 0 has a total capacity of 15.77 GiB of which 1.55 GiB is free. Including non-PyTorch memory, this process has 14.21 GiB memory in use. Of the allocated memory 11.89 GiB is allocated by PyTorch, and 1.95 GiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting max\_split\_size\_mb to avoid fragmentation. See documentation for Memory Management and PYTORCH\_CUDA\_ALLOC\_CONF



# TensorFlow vs PyTorch

 **Benchmark Comparison (Tesla V100 – 50K Image Inference)**

| Batch | TensorFlow<br>Throughput (img/s) | PyTorch Throughput<br>(img/s) | PyTorch vs TF Speed<br>Ratio | Top-1 Acc Diff |
|-------|----------------------------------|-------------------------------|------------------------------|----------------|
| 16    | 216 img/s                        | 767 img/s                     | 3.5× faster                  | +0.19 %        |
| 32    | 374                              | 828                           | 2.2× faster                  | +0.19 %        |
| 64    | 586                              | 838                           | 1.4× faster                  | +0.19 %        |
| 128   | 941                              | 860                           | 0.9× slower                  | +0.19 %        |
| 256   | 1327                             | 858                           | 0.6× slower                  | +0.19 %        |
| 512   | 1677                             | 842                           | 0.5× slower                  | +0.19 %        |
| 1024  | 1884                             | 807                           | 0.4× slower                  | +0.19 %        |

## ◆ 1. Low batch sizes ( $\leq 64$ )

PyTorch  $\gg$  TensorFlow

- PyTorch loads the model and batches more efficiently for small inputs.
- Likely using  **cudnn autotune + fused ops**  by default.
- TensorFlow suffers from graph-execution overhead for small batches.

## ◆ 2. Medium $\rightarrow$ large batch sizes ( $\geq 128$ )

TensorFlow  $\gg$  PyTorch

- TensorFlow scales extremely well once GPU is saturated.
- cuDNN kernels and graph optimization fully utilize the **V100 Tensor Cores**.
- PyTorch's eager execution hits memory fragmentation & dataloader sync overhead.

# Pretrained MobileNet GPU Run with TensorFlow on Python by Slurm

Akya Cuda'ya bağlanıp mobilenet\_gpu\_test.py yi sbatch ile bitişini sıraya ekleyerek çalıştırmak (min, average, max)

min → 1 GPU / 16 GB RAM / 10 CPU

| 🇫🇷 === Batch Size GPU Benchmark Results === |      |              |                |                    |                     |                      |               |               |  |  |  |  |
|---------------------------------------------|------|--------------|----------------|--------------------|---------------------|----------------------|---------------|---------------|--|--|--|--|
| Batch                                       | Size | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/sec) | Top-1 Acc (%) | Top-5 Acc (%) |  |  |  |  |
|                                             | 16   | 50000        | 256.24         | 0.0820             | 5.12                | 195.1                | 69.06         | 88.52         |  |  |  |  |
|                                             | 32   | 50000        | 139.94         | 0.0895             | 2.80                | 357.3                | 69.06         | 88.52         |  |  |  |  |
|                                             | 64   | 50000        | 85.24          | 0.1090             | 1.70                | 586.6                | 69.06         | 88.52         |  |  |  |  |
|                                             | 128  | 50000        | 51.36          | 0.1314             | 1.03                | 973.5                | 69.06         | 88.52         |  |  |  |  |
|                                             | 256  | 50000        | 35.71          | 0.1822             | 0.71                | 1400.0               | 69.06         | 88.52         |  |  |  |  |
|                                             | 512  | 50000        | 28.51          | 0.2909             | 0.57                | 1754.0               | 69.06         | 88.52         |  |  |  |  |
|                                             | 1024 | 50000        | 24.59          | 0.5019             | 0.49                | 2033.0               | 69.06         | 88.52         |  |  |  |  |
|                                             | 2048 | 50000        | 21.97          | 0.8790             | 0.44                | 2275.4               | 69.06         | 88.52         |  |  |  |  |
|                                             | 4096 | 50000        | 22.03          | 1.6945             | 0.44                | 2269.8               | 69.06         | 88.52         |  |  |  |  |

# Pretrained MobileNet GPU Run with TensorFlow on Python by Slurm

average → 2 GPU / 20 CPU / 32 GB RAM

| 🚦 === Batch Size GPU Benchmark Results === |              |                |                    |                     |                      |               |               |  |  |  |  |
|--------------------------------------------|--------------|----------------|--------------------|---------------------|----------------------|---------------|---------------|--|--|--|--|
| Batch Size                                 | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/sec) | Top-1 Acc (%) | Top-5 Acc (%) |  |  |  |  |
| 16                                         | 50000        | 224.09         | 0.0717             | 4.48                | 223.1                | 69.06         | 88.52         |  |  |  |  |
| 32                                         | 50000        | 129.12         | 0.0826             | 2.58                | 387.2                | 69.06         | 88.52         |  |  |  |  |
| 64                                         | 50000        | 82.43          | 0.1054             | 1.65                | 606.6                | 69.06         | 88.52         |  |  |  |  |
| 128                                        | 50000        | 50.30          | 0.1286             | 1.01                | 994.1                | 69.06         | 88.52         |  |  |  |  |
| 256                                        | 50000        | 33.71          | 0.1720             | 0.67                | 1483.2               | 69.06         | 88.52         |  |  |  |  |
| 512                                        | 50000        | 26.78          | 0.2732             | 0.54                | 1867.4               | 69.06         | 88.52         |  |  |  |  |
| 1024                                       | 50000        | 22.62          | 0.4615             | 0.45                | 2210.9               | 69.06         | 88.52         |  |  |  |  |
| 2048                                       | 50000        | 22.40          | 0.8958             | 0.45                | 2232.6               | 69.06         | 88.52         |  |  |  |  |
| 4096                                       | 50000        | 22.75          | 1.7504             | 0.46                | 2197.3               | 69.06         | 88.52         |  |  |  |  |

# Pretrained MobileNet GPU Run with TensorFlow on Python by Slurm

max → 4 GPU / 40 CPU / 64 GB RAM

📊 === Batch Size GPU Benchmark Results ===

| Batch Size | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/sec) | Top-1 Acc (%) | Top-5 Acc (%) |
|------------|--------------|----------------|--------------------|---------------------|----------------------|---------------|---------------|
| 16         | 50000        | 229.86         | 0.0736             | 4.60                | 217.5                | 69.06         | 88.52         |
| 32         | 50000        | 129.65         | 0.0829             | 2.59                | 385.7                | 69.06         | 88.52         |
| 64         | 50000        | 82.40          | 0.1054             | 1.65                | 606.8                | 69.06         | 88.52         |
| 128        | 50000        | 50.30          | 0.1286             | 1.01                | 994.1                | 69.06         | 88.52         |
| 256        | 50000        | 34.29          | 0.1749             | 0.69                | 1458.3               | 69.06         | 88.52         |
| 512        | 50000        | 28.37          | 0.2895             | 0.57                | 1762.4               | 69.06         | 88.52         |
| 1024       | 50000        | 23.74          | 0.4845             | 0.47                | 2106.0               | 69.06         | 88.52         |
| 2048       | 50000        | 23.23          | 0.9290             | 0.46                | 2152.8               | 69.06         | 88.52         |
| 4096       | 50000        | 22.38          | 1.7217             | 0.45                | 2233.9               | 69.06         | 88.52         |



# Min, Average, Max Karşılaştırma TensorFlow

| Batch Size | Throughput<br>(img/s) Min | Throughput<br>(img/s) Avg | Throughput<br>(img/s) Max |
|------------|---------------------------|---------------------------|---------------------------|
| 16         | 195.1                     | 223.1                     | 217.5                     |
| 32         | 357.3                     | 387.2                     | 385.7                     |
| 64         | 586.6                     | 606.6                     | 606.8                     |
| 128        | 973.5                     | 994.1                     | 994.1                     |
| 256        | 1400.0                    | 1483.2                    | 1458.3                    |
| 512        | 1754.0                    | 1867.4                    | 1762.4                    |
| 1024       | 2033.0                    | 2210.9                    | 2106.0                    |
| 2048       | 2275.4                    | 2232.6                    | 2152.8                    |
| 4096       | 2269.8                    | 2197.3                    | 2233.9                    |

# Pretrained MobileNet GPU Run with PyTorch on Python by Slurm

Akya Cuda'ya bağlanıp mobilenet\_gpu\_pytorch.py yi sbatch ile bitiğini sıraya ekleyerek çalıştırmak (min, average, max)

min → 1 GPU / 16 GB RAM / 10 CPU

## PyTorch GPU Benchmark Results

| Batch Size | Total Images | Total Time (s) | Avg Batch Time (s) | Avg Image Time (ms) | Throughput (img/s) | Top-1 Acc (%) | Top-5 Acc (%) |
|------------|--------------|----------------|--------------------|---------------------|--------------------|---------------|---------------|
| 16         | 50000        | 66.63          | 0.0213             | 1.33                | 750.5              | 69.25         | 88.81         |
| 32         | 50000        | 56.96          | 0.0364             | 1.14                | 877.9              | 69.25         | 88.81         |
| 64         | 50000        | 56.80          | 0.0726             | 1.14                | 880.3              | 69.25         | 88.81         |
| 128        | 50000        | 57.10          | 0.1460             | 1.14                | 875.6              | 69.25         | 88.81         |
| 256        | 50000        | 55.69          | 0.2841             | 1.11                | 897.8              | 69.25         | 88.81         |
| 512        | 50000        | 57.20          | 0.5836             | 1.14                | 874.2              | 69.25         | 88.81         |
| 1024       | 50000        | 57.68          | 1.1771             | 1.15                | 866.9              | 69.25         | 88.81         |

# Pretrained MobileNet GPU Run with PyTorch on Python by Slurm

average → 2 GPU / 20 CPU / 32 GB RAM

# Pretrained MobileNet GPU Run with PyTorch on Python by Slurm

max → 4 GPU / 40 CPU / 64 GB RAM

# Min, Average, Max Karşılaştırma PyTorch



# Python vs SLURM

```
module purge  
module load apps/truba-ai/gpu-2024.0
```

Bu modülde çoğu şey var ancak TensorRT ve Nvidia Nsights yok

Truba'nın dökümanı onları ayrı bir konteyner içinde çalıştırmamızı öneriyor  
kurulum yapınca yavaşlattığı için

## Kaynakça

<https://docs.truba.gov.tr/index.html>